



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии» (ИУ7)

## **ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №7**

**Название:** Обработка хеш-таблиц

**Дисциплина:** Типы и структуры данных

**Вариант:** 18

Студент

ИУ7-31Б

(Группа)

Постнов С.А

(Фамилия И. О.)

Преподаватель

Силантьева А.В.

## Оглавление

|                                                                                                                                                              |   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| Условие задачи .....                                                                                                                                         | 3 |
| Описание ТЗ.....                                                                                                                                             | 3 |
| 1. Описание входных данных .....                                                                                                                             | 3 |
| 2. Описание результатов работы программы .....                                                                                                               | 3 |
| 3. Способ обращения к программе .....                                                                                                                        | 3 |
| 4. Задача программы .....                                                                                                                                    | 4 |
| 5. Описание возможных аварийных ситуаций и ошибок пользователя....                                                                                           | 4 |
| Описание внутренних структур данных .....                                                                                                                    | 4 |
| Описание алгоритма работы .....                                                                                                                              | 6 |
| Сравнение эффективности поиска слова в двоичном дереве поиска,<br>сбалансированном двоичном дереве поиска и хеш-таблице в зависимости<br>от размерности..... | 6 |
| Выводы .....                                                                                                                                                 | 7 |
| Ответы на контрольные вопросы .....                                                                                                                          | 7 |

## **Условие задачи**

Построить хеш-таблицу для зарезервированных слов языка C++ (не менее 20 слов), содержащую HELP для каждого слова. Выдать на экран подсказку по введенному слову. Выполнить программу для различных размерностей таблицы и сравнить время поиска и количество сравнений. Для указанных данных создать сбалансированное дерево. Добавить подсказку по вновь введенному слову, используя при необходимости реструктуризацию таблицы.

## **Описание ТЗ**

### **1. Описание входных данных**

#### **Ограничения:**

1. Любой код, не лежащий в диапазоне от 0 до 6 включительно, считается ошибочным

### **2. Описание результатов работы программы**

В результате работы программы создается двоичное дерево. После этого выводится меню программы, содержащее в себе следующие пункты:

- 0) Выход
- 1) Вывести сбалансированное дерево зарезервированных слов
- 2) Вывести хеш-таблицу для зарезервированных слов
- 3) Получить подсказку по введенному слову
- 4) Добавить подсказку по введенному слову в хеш-таблицу
- 5) Сравнить время поиска и кол-во сравнений для различных размерностей таблицы
- 6) Изменить тип хеширования

### **3. Способ обращения к программе**

Обращение к программе происходит из командной строки путем запуска исполняемого файла «*app.exe*».

## 4. Задача программы

Задача данной программы заключала в себе несколько составных частей:

- Построение сбалансированного дерева поиска для слов, построение хеш-таблиц различного вида (закрытые и открытые), добавление и поиск в хеш-таблице
- Сравнение времени поиска слова в двоичном дереве поиска, сбалансированном двоичном дереве поиска и хеш-таблице

## 5. Описание возможных аварийных ситуаций и ошибок пользователя

- Некорректный код меню:
  - Код возврата: 1
  - Сообщение: «Некорректный код. Попробуйте снова.»
- Ошибка выделения памяти:
  - Код возврата: 2
  - Сообщение: «Не удалось выделить память. Попробуйте снова.»
- Ошибка при чтении слова или подсказки при добавлении или поиске:
  - Код возврата: 3
  - Сообщение: «Не удалось получить элемент. Попробуйте снова.»

## Описание внутренних структур данных

Тип данных, описывающий структуру для работы с элементами хеш-таблицы при закрытом хешировании:

```
typedef struct
{
    char keyword[MAX_KEYWORD_LEN + 1]; // Ключевое слово
    char help[MAX_HELP_LEN + 1];      // Подсказка к слову
} keyword_info_t;
```

Тип данных, описывающий структуру для работы с элементами хеш-таблицы при открытом хешировании:

```
typedef struct list_keyword
{
```

```

char keyword[MAX_KEYWORD_LEN + 1]; // Ключевое слово
char help[MAX_HELP_LEN + 1];      // Подсказка к слову

struct list_keyword *next;         // Указатель на следующий элемент
} list_keyword_info_t;

```

Тип данных, описывающий структуру для работы с хеш-таблицей при различном хешировании:

```

typedef struct
{
    void *data; // Данные
    int size;   // Размер таблицы
} hash_table_t;

```

Тип данных, описывающий структуру для работы со сбалансированным двоичным деревом поиска:

```

typedef struct balance_tree_node
{
    char *keyword; // Ключевое слово
    int height;    // Максимальная высота

    struct balance_tree_node *left; // Указатель на левого потомка
    struct balance_tree_node *right; // Указатель на правого потомка
} balance_tree_node_t;

```

Тип данных, описывающий структуру для работы с двоичным деревом поиска:

```

typedef struct tree_node
{
    char *value; // Ключевое слово

    struct tree_node *left; // Указатель на левого потомка
    struct tree_node *right; // Указатель на правого потомка
} tree_node_t;

```

Тип данных, описывающий структуру для хранения результатов замерного эксперимента:

```

typedef struct measure
{
    long long unsigned time; // Время
    char type;               // Тип данных
    int count;               // Кол-во элементов
    int mem;                 // Память
} measurement_table;

```

## Описание алгоритма работы

1. Пользователю выводится меню программы с возможностью выбрать желаемый
2. Пользователь вводит желаемый пункт меню
3. Пока пользователь не введет «0» (пункт выхода из программы), ему будет предложено вводить номера меню и выполнять желаемые действия

## Сравнение эффективности поиска слова в двоичном дереве поиска, сбалансированном двоичном дереве поиска и хеш-таблице в зависимости от размерности

Полученная таблица замеров при поиске случайного элемента в двоичном дереве поиска, сбалансированном двоичном дереве поиска и хеш-таблице (закрытое хеширование) при 10000 запусков для каждого типа данных:

| Тип         | Время / Память<br>(10 элементов,<br>мкс) | Время / Память<br>(50 элементов,<br>мкс) | Время / Память<br>(100 элементов,<br>мкс) |
|-------------|------------------------------------------|------------------------------------------|-------------------------------------------|
| ДДП         | 45 / 320                                 | 51 / 1600                                | 45 / 3200                                 |
| АВЛ-дерево  | 61 / 400                                 | 54 / 2000                                | 54 / 4000                                 |
| Хеш-таблица | 206 / 1686                               | 311 / 8366                               | 316 / 16716                               |

Полученная таблица замеров при поиске случайного элемента в двоичном дереве поиска, сбалансированном двоичном дереве поиска и хеш-таблице (открытое хеширование) при 10000 запусков для каждого типа данных:

| Тип        | Время / Память<br>(10 элементов,<br>мкс) | Время / Память<br>(50 элементов,<br>мкс) | Время / Память<br>(100 элементов,<br>мкс) |
|------------|------------------------------------------|------------------------------------------|-------------------------------------------|
| ДДП        | 46 / 320                                 | 51 / 1600                                | 46 / 3200                                 |
| АВЛ-дерево | 45 / 400                                 | 51 / 2000                                | 70 / 4000                                 |

|                    |            |            |             |
|--------------------|------------|------------|-------------|
| <b>Хеш-таблица</b> | 180 / 1776 | 252 / 8816 | 261 / 17616 |
|--------------------|------------|------------|-------------|

## Выводы

В ходе работы были изучены способы и алгоритмы работы с хеш-таблицами и деревьями, а также проведены замерные эксперименты, по результатам которых можно сделать вывод, что реализованный алгоритм выполняет поиск элемента быстрее всего в ДДП и АВЛ-дереве, так как их отсортированность позволяет сэкономить ресурсы и дополнительные обходы большого количества элементов, в то время как в хеш-таблице задействуется хеш-функция, имеющая сложность  $O(n)$ . Таким образом, деревья выигрывают во времени у хеш-таблицы в 4-5 раз.

## Ответы на контрольные вопросы

- 1) Для каждой вершины АВЛ-дерева высота двух поддеревьев различается не более, чем на единицу. Однако в таком дереве количество вершин в левом и правом поддеревьях может отличаться больше, чем на единицу, что противоречит условию идеальной сбалансированности
- 2) Принципиальных отличий в поиске элементов в таких деревьях нет
- 3) Хеш-таблица — это специальная структура данных для хранения пар ключей и их значений. Ассоциативный массив, в котором ключ представлен в виде хеш-функции. Построение таблицы происходит следующим образом:
  - Первоначально ключ передается в хеш-функцию, которая на выходе дает индекс ячейки в таблице
  - Элемент размещается по полученному индексу
  - В случае возникновения *коллизий* происходит их устранение или реструктуризация таблицы
- 4) Может возникнуть ситуация, когда разным ключам соответствует

одно значение хеш-функции, то есть, когда  $h(K1) = h(K2)$ , в то время как  $K1 \neq K2$ . Такая ситуация называется *коллизией*. Пути разрешения коллизии:

- **Открытое хеширование** – в случае, когда элемент таблицы с индексом, который вернула хеш-функция, уже занят, к нему присоединяется связный список. Таким образом, если для нескольких различных значений ключа возвращается одинаковое значение хеш-функции, то по этому адресу находится указатель на связанный список, который содержит все значения
- **Закрытое хеширование** – в этом случае, если ячейка с вычисленным индексом занята, то можно просто просматривать следующие записи таблицы по порядку (с шагом 1), до тех пор, пока не будет найден ключ  $K$  или пустая позиция в таблице. При этом, если индекс следующего просматриваемого элемента определяется добавлением какого-то постоянного шага (от 1 до  $n$ ), то данный способ разрешения коллизий называется линейной адресацией

5) Хеш-таблицы становятся неэффективными при большом количестве конфликтов (коллизий)

6) Эффективность поиска:

- В ДДП:  $O(\log(N))$
- В АВЛ-дереве:  $O(\log(N))$
- В Хеш-таблице:  $O(1)$